cognee

Moss

MEMORY CONTROL AGENT

TRUSTED BY ENGINEERS IN

Software

splunk> Redis aws Infosys botpress studocu kovai.co

Blended ATLASSIAN AUTODESK ribbon REPLY rentpost

eCommerce

Alibaba

Hardware

intel

Finance

QRT DBS

Royal Bank of Canada ALTUM

Energy

raizen

PEERS Consulting + Technology

Academia

TECHNISCHE UNIVERSITÄT BERLIN

Berkeley UNIVERSITY OF CALIFORNIA

GESIS Leibniz-Institut für Sozialwissenschaften

BROWN UNIVERSITY

PUC RIO

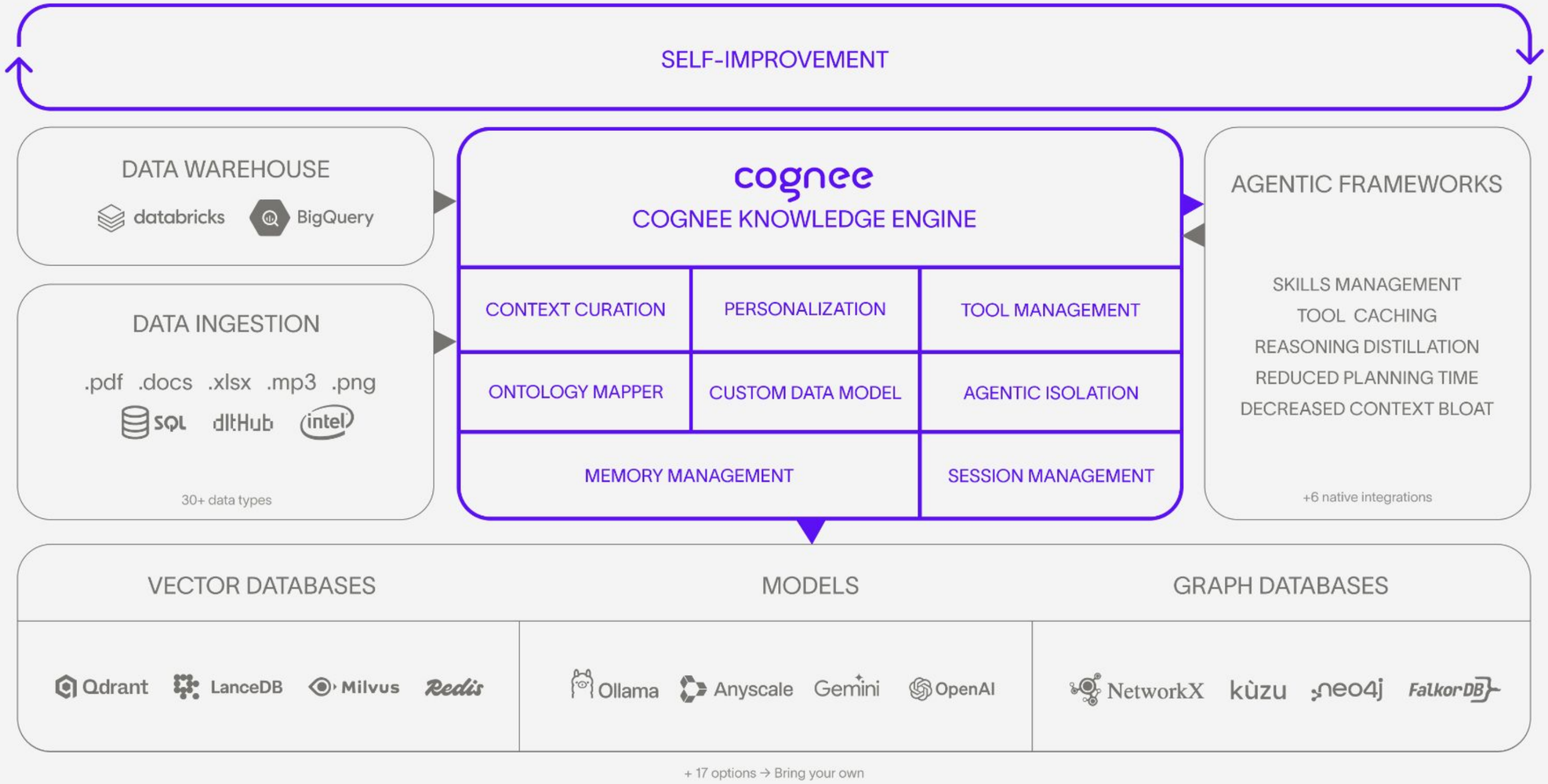
+80 other companies and thousands of open source developers

★ 16.8k

SDK runs/mo 4M+

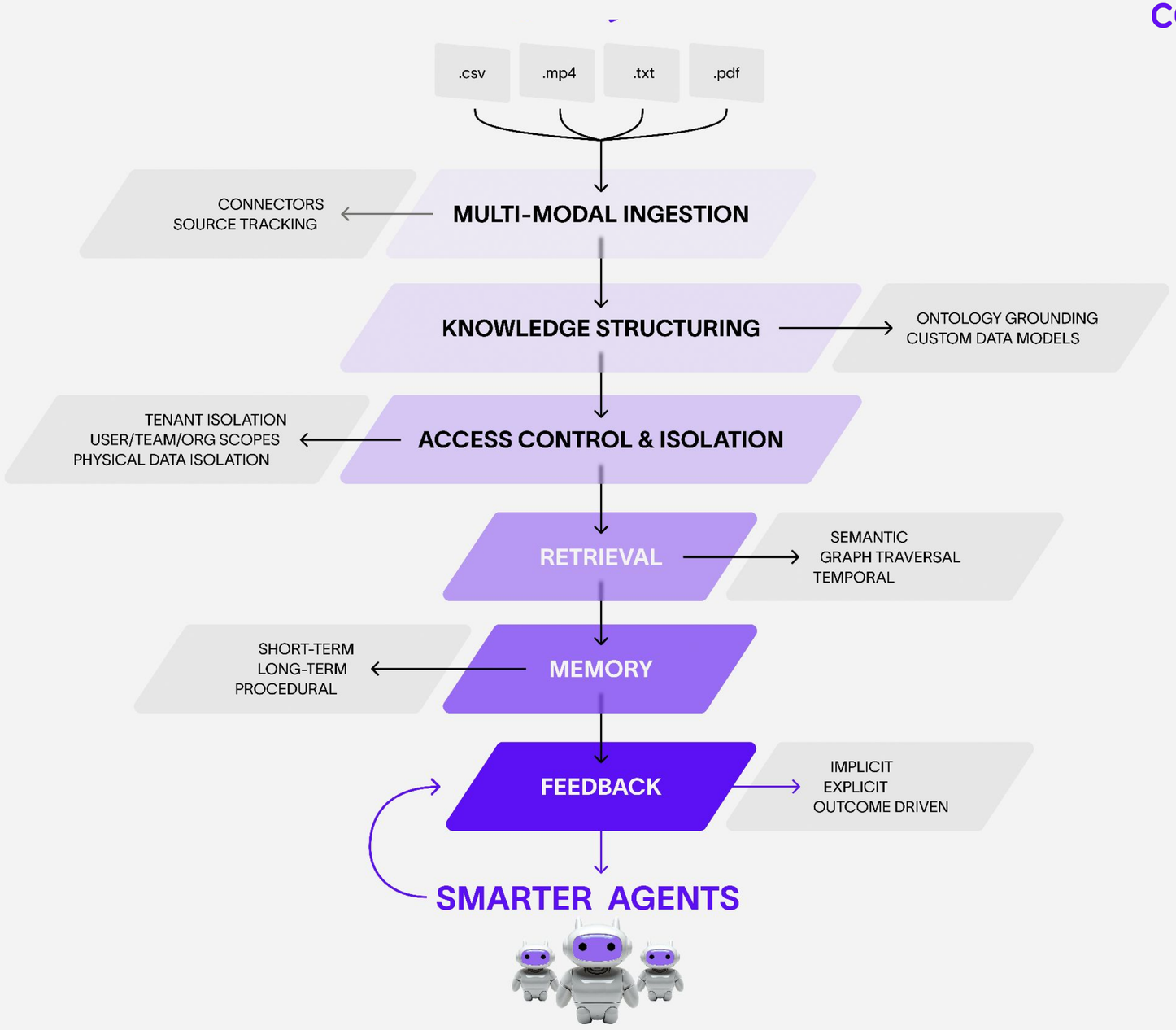
cognee

WE ARE THE ENGINE AT THE CORE OF YOUR AI STACK



cognee

From Data Ingestion To Smarter Agents With Cognee



SELF-IMPROVING SKILLS FOR REMOTE AGENTS

A pipeline where four sandboxed agents share memory —
and the skill they use improves itself.

THE LAYER

Cognee is the shared memory.

Code, docs, agent findings, and skill files — all ingested into a knowledge graph the next agent can query.

WHAT HAPPENS

Each sandbox writes to local disk at `/var/cognee-state` .

After every agent finishes, the script tars that state into the shared volume. The next agent restores it.

GRAPH

Cognee

Datasets, pipeline runs, nodes, edges.

VECTORS

Moss

Vector indexes that survive across sandboxes.

RUNTIME

Daytona

Disposable sandboxes + one shared volume.

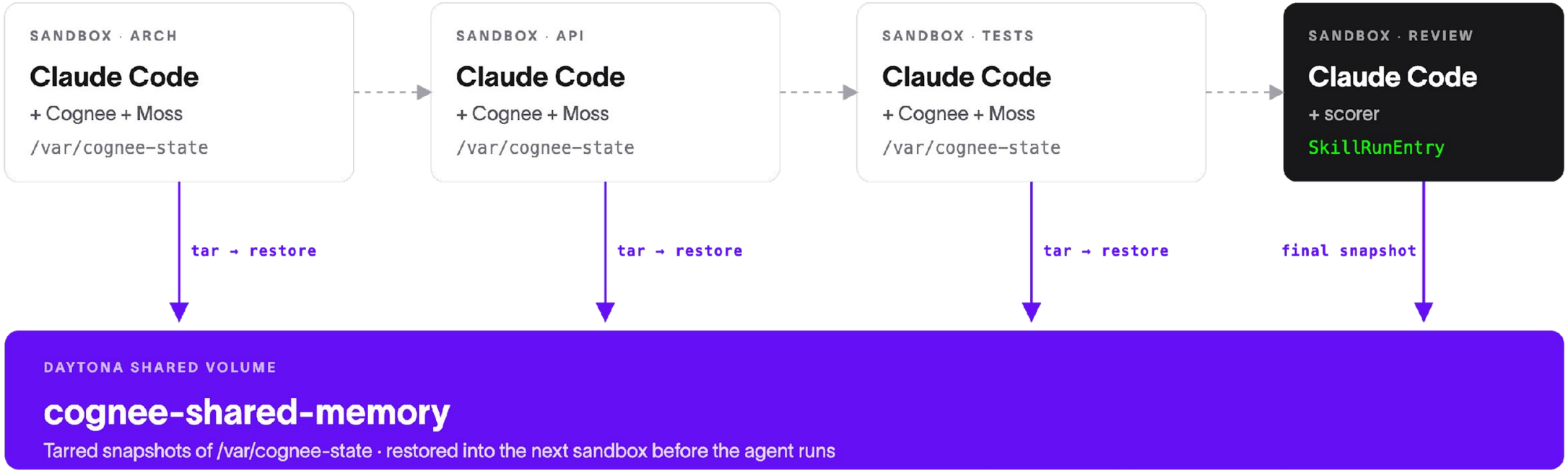
AGENT

Claude Code

With the cognee-memory plugin installed.

ARCHITECTURE

One volume. Four sandboxes. One graph.



VECTOR LAYER

Moss

Indexes available across all four sandboxes — no snapshot needed.

SKILL

/opt/cognee-demo-skills/code-review/SKILL.md

Ingested via cognee.remember() — lives in the graph, not on disk.

THE PIPELINE

Four agents and one graph that grows between them

01
arch

Investigates the target repo. Writes structural findings into the graph.

02
api

Recalls arch's findings. Adds API surface and dependencies.

03
tests

Sees both above. Adds test coverage and gaps.

04
review

Uses the bundled skill. Then scores the skill itself.

- Each agent recalls memory before acting
- Each agent stores findings via /cognee-memory:cognee-remember

THE ENTIRE PIPELINE — ONE COMMAND

From repo root.

```
# load env set -a [ -f .env ] && . ./.env set +a # run the pipeline python
distributed/deploy/daytona_onboarding_demo.py \   --repo https://github.com/topoteretes/cognee \
--cognee-install-spec git+...@graphskills-on-agentic
```

- DAYTONA_API_KEY
- ANTHROPIC_API_KEY
- LLM_API_KEY
- MOSS_PROJECT_ID
- MOSS_PROJECT_KEY

Five env vars. One script. Four agents. One shared graph.

Skills aren't shipped as code. They're stored as memory.

WRITTEN AS

```
/opt/cognee-demo-skills/
  └─ code-review/
       └─ SKILL.md
```

Plain markdown — written by hand, dropped into the sandbox.

INGESTED AS

```
cognee.remember(
    "/opt/cognee-demo-skills",
    dataset_name="shared_codebase_memory",
    node_set=["skills"],
    self_improvement=False,
)
```

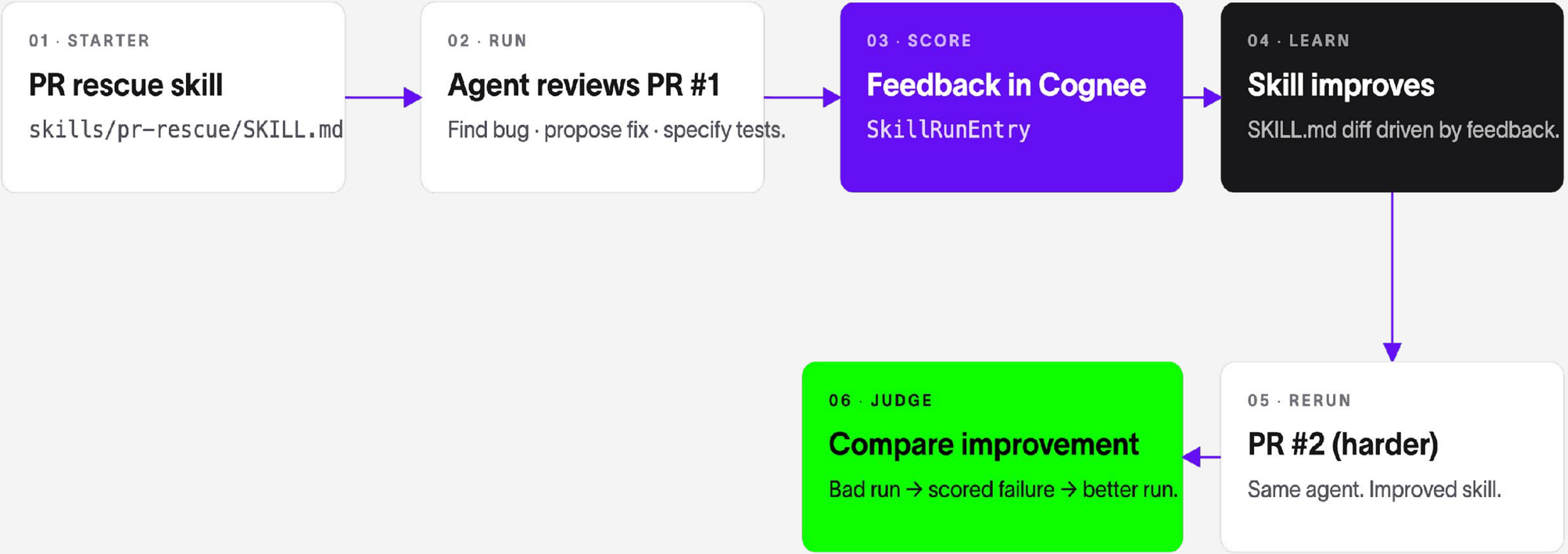
After this call the skill lives in the graph. Agents discover it by searching memory — same way they find anything else.

THE FEEDBACK LOOP

The reviewer scores its own skill before the sandbox dies.

THE CORE LOOP

Score the run. Improve the skill. Run it again.



WHAT TEAMS SHIP

Four artifacts. That's the deliverable.

01

The skill

skills/pr-rescue/SKILL.md

02

Helper rubric

Optional scorer that turns review output into a SkillRunEntry.

03

Run logs

Before / after, with the failure mode visible.

04

Skill diff

The change to SKILL.md driven by feedback.

SCORING

100 points. Most of them on the loop.

40 — **PR rescued**

Bug found, fix correct, tests pass or are well specified.

25 — **Self-improvement**

Skill changes are driven by recorded feedback.

20 — **Review quality**

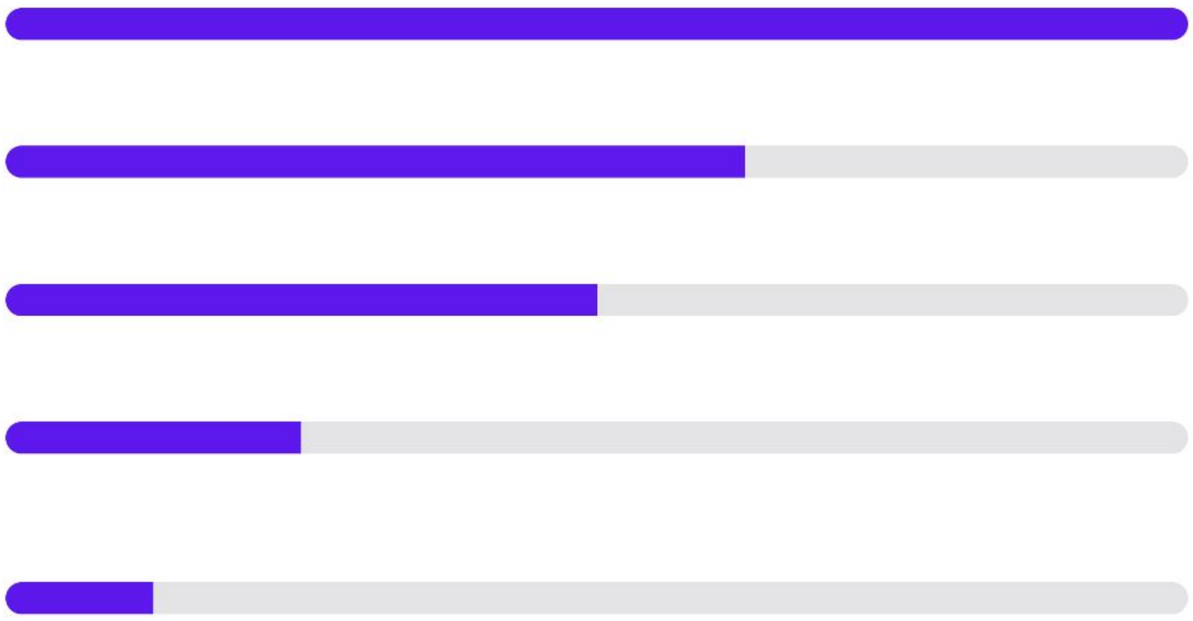
Clear severity, file references, impact, fix, tests.

10 — **Reproducibility**

Daytona run works cleanly.

5 — **Safety**

No unrelated changes. No permission bypasses. No hallucinated APIs.



A new broken PR drops. Your improved skill must handle it without manual edits during the round.

https://github.com/topoteretes/cognee-daytona-moss-hackathon

Get the link for your keys:

https://send.bitwarden.com/#hFEfKcEk4kC1KrQ3ATYDqA/Q7dYaCGmOoyX47uvoY-3CA